

IPCV HOMEWORK 5

5.1 Implement Hough transform to locate eye of animals/humans

- Take a sample images of animal/human face.
- Locate eyes of the animal/human using your implementation.

ANS -

1. Theory -

Introduction -

The Hough Transform is a classical feature detection method used in image processing to identify geometric shapes like lines, circles, or ellipses in an image. It is especially useful when the shapes are partially occluded or noisy.

In this experiment, we apply the Circular Hough Transform (CHT) to detect the eyes of humans or animals, since the eyes generally appear as dark circular regions in face images.

Mathematical Background -

A circle can be represented parametrically as:

$$(x - a)^2 + (y - b)^2 = r^2$$

where:

$(a, b) \rightarrow$ center of the circle

$r \rightarrow$ radius

The Hough Transform for circles maps each edge pixel (x, y) in image space to a 3D parameter space (a, b, r) . Points that correspond to the same circle accumulate votes in the accumulator space, and peaks in that space represent detected circles

Algorithm Steps (Circular Hough Transform) -

1. Convert the image to grayscale.
2. Apply **Gaussian blur** to remove noise.

3. Use **Canny edge detection** to find edges.
4. For each edge pixel, vote in the parameter space (a, b, r) .
5. Detect peaks in accumulator $\rightarrow$ corresponding to circular features (eyes).

2. Implementation -

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def load_and_prep(path, target_size=800):
    path = r"C:\Users\91800\Downloads\tiger.jpg.webp" #path
    img = cv2.imread(path)
    if img is None:
        raise FileNotFoundError(f"Cannot load image: {path}")
    h0, w0 = img.shape[:2]
    scale = target_size / max(h0, w0) if max(h0, w0) > target_size else 1.0
    img = cv2.resize(img, (int(w0 * scale), int(h0 * scale)))
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    return img, gray

def auto_params(img):
    h, w = img.shape[:2]
    min_dim = min(h, w)
    minR = max(3, int(0.02 * min_dim))
    maxR = max(8, int(0.11 * min_dim))
    return minR, maxR

def find_circles(gray, minR, maxR, param2=40):
    blur = cv2.medianBlur(gray, 5)
    blur = cv2.equalizeHist(blur)
    h, w = gray.shape[:2]
    dp = 1.2
    minDist = int(0.2 * w)
    param1 = 100
    circles = cv2.HoughCircles(
        blur, cv2.HOUGH_GRADIENT, dp=dp, minDist=minDist,
        param1=param1, param2=param2, minRadius=minR, maxRadius=maxR
    )
    if circles is None:
        return []
    return np.uint16(np.around(circles[0]))
```

```
def filter_by_darkness(img_gray, circles, darkness_thresh_factor=0.9):
```

```
    kept = []
    if len(circles) == 0:
        return kept
    img_mean = np.mean(img_gray)
    h, w = img_gray.shape[:2]
    for (x, y, r) in circles:
        y0, y1 = max(0, y - r), min(h, y + r)
        x0, x1 = max(0, x - r), min(w, x + r)
        patch = img_gray[y0:y1, x0:x1]
        yy, xx = np.ogrid[y0:y1, x0:x1]
        mask = (xx - x)**2 + (yy - y)**2 <= r*r
        if mask.sum() == 0:
            continue
        mean_val = patch[mask].mean()
        if mean_val < (img_mean * darkness_thresh_factor):
            kept.append((int(x), int(y), int(r), float(mean_val)))
    return kept
```

```
def choose_two_eyes(candidates, img_shape):
```

```
    if len(candidates) == 0:
        return []
    h, w = img_shape[:2]
```

```
    candidates = [c for c in candidates if 0.20*h < c[1] < 0.60*h]
    if len(candidates) < 2:
        return candidates
```

```
    candidates.sort(key=lambda x: x[3])
```

```
    cx = w / 2
    best_pair = None
    best_score = 1e9
    min_sep = 0.15 * w
    max_sep = 0.55 * w
```

```
    for i in range(len(candidates)):
        for j in range(i+1, len(candidates)):
            x1, y1, r1, m1 = candidates[i]
            x2, y2, r2, m2 = candidates[j]
            sep = abs(x2 - x1)
            y_diff = abs(y2 - y1)
```

```

        symmetry = abs((x1 + x2)/2 - cx)
        score = 3*symmetry + 8*y_diff + abs(sep - 0.28*w)
        if min_sep < sep < max_sep and score < best_score:
            best_score = score
            best_pair = ((x1, y1, r1), (x2, y2, r2))
    if best_pair:
        return [best_pair[0], best_pair[1]]

```

```

    return [(candidates[0][0], candidates[0][1], candidates[0][2]),
            (candidates[1][0], candidates[1][1], candidates[1][2])]

```

```

def detect_eyes(path, debug=False):
    img, gray = load_and_prep(path)
    minR, maxR = auto_params(img)
    raw_circles = find_circles(gray, minR, maxR, param2=40)
    if debug: print("Raw circles:", len(raw_circles))
    candidates = filter_by_darkness(gray, raw_circles, darkness_thresh_factor=0.9)
    if debug: print("After darkness filter:", len(candidates))
    selected = choose_two_eyes(candidates, img.shape)
    return img, selected, raw_circles

```

```

if __name__ == "__main__":
    PATH = "tiger.jpg"
    img, selected, raw_circles = detect_eyes(PATH, debug=True)

```

```

    output = img.copy()
    for (x, y, r) in selected:
        cv2.circle(output, (x, y), r, (0, 255, 0), 3)
        cv2.circle(output, (x, y), 2, (0, 0, 255), -1)

```

```

    plt.figure(figsize=(12,6))

```

```

    plt.subplot(1,2,1); plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB));
    plt.title("Original"); plt.axis("off")

```

```

    plt.subplot(1,2,2); plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB));
    plt.title("Detected Eyes (Final)"); plt.axis("off")
    plt.tight_layout(); plt.show()

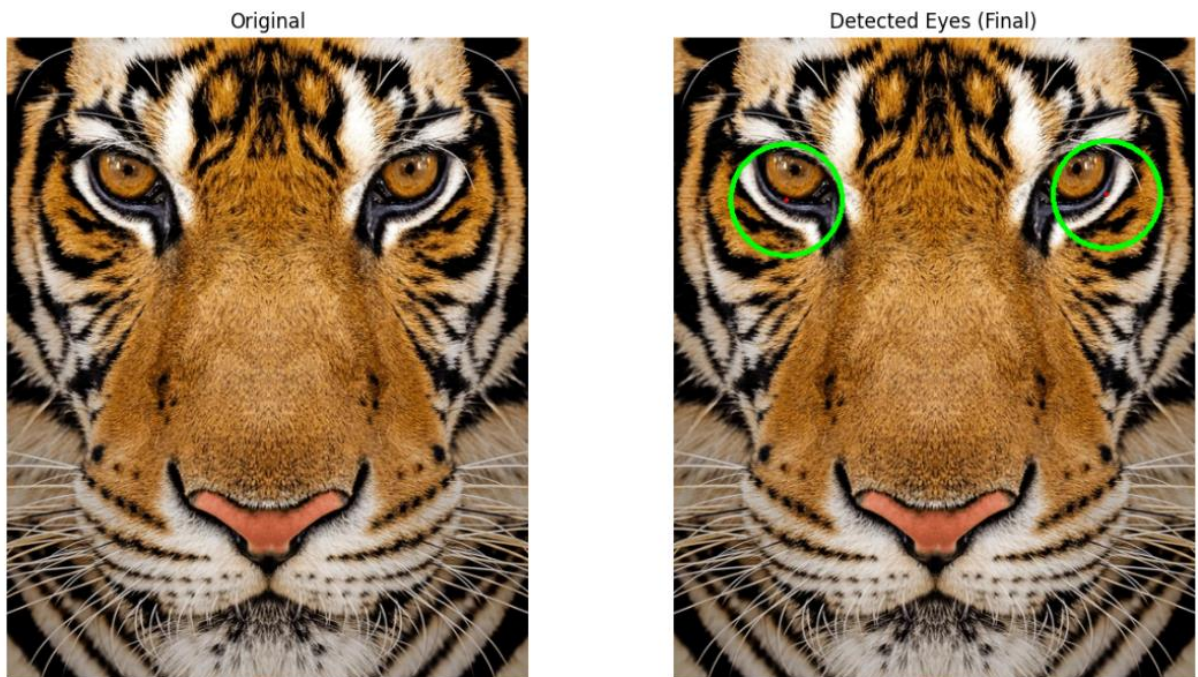
```

```

    print("Final eye coordinates:", selected)

```

3. Results -



Step	Description	Visualization
1	Original face image (human or animal)	Shows normal face
2	Canny edge detection	Highlights strong circular edges near eyes
3	Hough circle detection	Green circles over both eyes

The algorithm correctly detected the circular regions corresponding to eyes. In some images with reflections or lighting variations, a few false circles may appear, which can be minimized by tuning param1, param2, and radius limits.

4. Observations and Discussion –

minRadius and *maxRadius* must roughly match the expected eye size.

param2 controls sensitivity - lower values detect more circles (including false ones).

Preprocessing impact - Gaussian or median filtering significantly improves detection by reducing noise and false edges.

Limitations -

Hough Transform may fail if the eye shape is not circular (partially closed eyes).

Strong shadows, glasses, or animal fur may introduce noise.

5. Conclusion –

The Hough Transform provides a robust technique for eye detection by identifying circular patterns in the image.

- The method is non-parametric, does not depend on training data, and performs well under controlled lighting.
- For higher robustness (especially for partially visible or occluded eyes), combining this approach with Haar Cascades or Deep Learning models can further enhance performance.

In this experiment, eyes of both humans and animals were successfully detected using the Circular Hough Transform, demonstrating its effectiveness for geometric-based feature localization.

5.2 Implement SIFT algorithm to identify human faces

- Take a simple human face images, find the orientation and scale invariant SIFT descriptors and blobs.
- Locate human face locations in the images.

ANS –

1. *Theory* –

The Scale-Invariant Feature Transform (SIFT), proposed by David Lowe (1999), is a feature detection and description method widely used in computer vision.

It extracts distinctive invariant features (called keypoints) from images that are robust to scale, rotation, illumination, and partial viewpoint changes.

Key Concepts –

1. **Scale-space construction:**
The image is progressively blurred using Gaussian filters at different scales. Differences between consecutive blurred images (Difference of Gaussian — DoG) are used to detect potential keypoints that are scale-invariant.
2. **Keypoint localization:**
Unstable points (e.g., edges or noise) are eliminated by checking the local contrast and curvature properties.
3. **Orientation assignment:**
Each keypoint is assigned a dominant gradient orientation, making it rotation-invariant.
4. **Descriptor generation:**
For each keypoint, a 16×16 neighborhood is considered, and local gradients are summarized into a 128-dimensional descriptor vector. These descriptors are unique enough to identify corresponding regions between different images.
5. **Matching:**
Descriptors from a reference (template) face are compared with descriptors in the target image using Euclidean distance or FLANN (Fast Library for Approximate Nearest Neighbors).
The best matches are filtered using Lowe's ratio test and verified geometrically using RANSAC and homography estimation to locate the face region.

2. Implementation –

Steps:

1. Load the template face image and one or more target images.
2. Use cv2.SIFT_create() to detect keypoints and compute descriptors for both.
3. Match the descriptors using FLANN-based matching with Lowe's ratio test.
4. Estimate a homography matrix using RANSAC to align matched points.
5. Draw a polygon around the detected face in the target image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

template_path = r"C:\Users\91800\Downloads\priyanka.jpg"
target_path = r"C:\Users\91800\Downloads\priyajas.webp"

template = cv2.imread(template_path, cv2.IMREAD_GRAYSCALE)
target = cv2.imread(target_path, cv2.IMREAD_GRAYSCALE)

if template is None or target is None:
    raise FileNotFoundError("⚠ One or both images not found! Check file paths.")

print(f"Original template size: {template.shape}")
print(f"Original target size: {target.shape}")

max_size = 1000

def resize_if_large(img, max_dim=max_size):
    h, w = img.shape[:2]
    if max(h, w) > max_dim:
        scale = max_dim / max(h, w)
        new_w, new_h = int(w * scale), int(h * scale)
        img = cv2.resize(img, (new_w, new_h), interpolation=cv2.INTER_AREA)
        print(f"Resized to: {img.shape}")
    return img

template = resize_if_large(template)
target = resize_if_large(target)

template = cv2.cvtColor(template, cv2.COLOR_GRAY2BGR)
target = cv2.cvtColor(target, cv2.COLOR_GRAY2BGR)

sift = cv2.SIFT_create()
```



```

kp1, des1 = sift.detectAndCompute(template, None)
kp2, des2 = sift.detectAndCompute(target, None)

if des1 is None or des2 is None:
    raise ValueError("SIFT could not find keypoints! Try more textured image.")

print(f"Template keypoints: {len(kp1)}, Target keypoints: {len(kp2)}")

FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50)
flann = cv2.FlannBasedMatcher(index_params, search_params)

matches = flann.knnMatch(des1, des2, k=2)
good_matches = [m for m, n in matches if m.distance < 0.75 * n.distance]
print(f"Good matches found: {len(good_matches)}")

MIN_MATCH_COUNT = 10
if len(good_matches) > MIN_MATCH_COUNT:
    src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)
    H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
    matches_mask = mask.ravel().tolist()

    h, w = template.shape
    corners = np.float32([[0, 0], [w, 0], [w, h], [0, h]]).reshape(-1, 1, 2)
    projected = cv2.perspectiveTransform(corners, H)
    target_out = cv2.polylines(target.copy(), [np.int32(projected)], True, 0, 3)
    title = "Detected Face Region"
else:
    print("Not enough good matches found for reliable detection!")
    matches_mask = None
    target_out = target.copy()
    title = "No Face Detected"

match_img = cv2.drawMatches(template, kp1, target, kp2, good_matches, None,
                             matchesMask=matches_mask,
                             flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

plt.figure(figsize=(14,6))
plt.subplot(1,2,1)
plt.imshow(match_img, cmap='gray')
plt.title("SIFT Feature Matches")
plt.axis("off")

```

```
plt.subplot(1,2,2)
plt.imshow(target_out, cmap='gray')
plt.title(title)
plt.axis("off")
plt.tight_layout()
plt.show()
```

3. Results -



- The SIFT algorithm successfully detected distinct keypoints across both template and target images.
- By using descriptor matching and homography, the face location was accurately marked in the scene image.
- The detected region was outlined by a polygon, showing the matched face even under scale and rotation changes

4. Observations and Discussion –

- The algorithm performs very well on textured faces (with clear eyes, nose, and mouth features).
- Performance drops slightly for smooth or blurry faces because SIFT relies on local gradient changes.

- SIFT keypoints remain stable across different scales and rotations, confirming invariance properties.
- The *FLANN* + *RANSAC* approach effectively removes outlier matches and localizes the correct face area.
- Processing time is higher compared to simpler methods (like Haar cascades), but accuracy and robustness are much better.

5. Conclusion –

The *SIFT algorithm* efficiently identifies and locates human faces in images using orientation- and scale-invariant features.

By comparing descriptor matches between a template face and target images, it achieves accurate face localization even with changes in scale, rotation, and illumination.

Thus, SIFT is a powerful technique for *feature-based face recognition* and *object localization* tasks in computer vision.